06 POINTERS AND REFERENCES • THEORY

05 references and pointer comparison

Detailed Theory, Exact C++ Rules, Memory Models & Exam Reference

05 — References (`T&`) and Pointers vs. References Comparison

1. Definition of a Reference

A **reference** (`T&`) is an **alias** (an alternative name) for an already-existing object. Once initialized to refer to a variable, every operation performed on the reference acts directly on the original variable.

CPP

```cpp
int x = 10;
int& ref = x; // ref is an alias for x

ref = 25;     // Modifies x directly! x is now 25
cout << (&x == &ref); // Prints 1 (true) — ref has the exact same address as x
```

2. Strict Rules of References in C++

1. **Mandatory Initialization**: A reference **must** be initialized when declared. Writing `int& ref;` is a **Compilation Error**.
2. **Cannot Be Rebound**: Unlike a pointer, a reference cannot be made to refer to a different variable after initialization. If you write:

   CPP

   ```cpp
   int a = 10, b = 20;
   int& ref = a;
   ref = b; // Does NOT rebind ref to b! It assigns the value of b (20) into a!
   ```

3. **No Null References**: There is no such thing as a valid "null reference" (`int& r = *nullptr;` is Undefined Behavior).
4. **No References to References or Arrays of References**: You cannot declare `int& arr[5]` or a pointer to a reference `int&* p` because references are aliases, not distinct addressable objects in the C++ type system (`[dcl.ref]`).
5. `const` **References (`const T&`) Can Bind to Temporaries (Rvalues)**:
   - A non-const lvalue reference `int& r = 50;` fails to compile.
   - A `const` lvalue reference `const int& r = 50;` is **valid** and extends the lifetime of the temporary value `50` to match the scope of `r` .

3. Comprehensive Comparison: Pointers vs. References (Exam Table)

Feature	Pointer (`int* p`)	Reference (`int& ref`)
Core Concept	Stores the memory address of an object	Acts as an alias (another name) for an existing object
Declaration without Initialization	Allowed (`int* p;` — though creates a wild pointer)	**Compilation Error** (`int& ref;` is illegal)
Rebinding / Reassignment	Can be reassigned to point to different objects (`p = &y;`)	**Cannot** be rebound; stays bound to its initial target for life
Nullability	Can be `nullptr`	Cannot be null in a well-defined program
Dereferencing Syntax	Requires explicit `*p` or `p→member`	Automatic; used directly like a normal variable (`ref` , `ref.member`)
Address-of Operator (`&`)	`&p` gives the address of the pointer variable itself	`&ref` gives the address of the referred-to target variable
Indirection Levels	Supports multi-level pointers (`int**` , `int**&`)	Single level only (reference to reference collapses to `int&`)
Pointer Arithmetic	Supported (`p++` , `p + 1` , `p - q`)	Not applicable (`ref++` increments the target's value)
Arrays	Array of pointers is legal (`int* arr[10];`)	Array of references is **illegal** (`int& arr[10];` fails)
When to Prefer	Optional parameters (`nullptr`), dynamic arrays, rebindable links	Function parameters (`const T&` , `T&`), operator overloading (`operator+` , `operator<<`)